Akamata Handbook — 15-minute tour

Akamata is a Zig 0.16 web framework that targets two deploy shapes from one source: a native binary (VPS / Cloudflare Containers) and a Cloudflare Workers wasm module. The DB layer abstracts SQLite, Turso (libsql), and Cloudflare D1 behind a URL — your handler code doesn't know which one is live.

This page covers everything you need to ship a CRUD API. Each section is ~2 minutes; skim or skip as needed.

---

0. Install

```
git clone https://github.com/yourorg/Akamata
cd Akamata
zig build cli      # ./zig-out/bin/akamata
# (optional) put zig-out/bin on PATH so `akamata` works from any dir
```

You'll also want:

- `node` + `wrangler` (for Workers deploy and local `wrangler dev`)
- `turso` CLI (only if you target Turso)
- `docker` (only if you target Cloudflare Containers)

---

1. Scaffold a project (30 seconds)

```
akamata init mynotes --target=both
cd mynotes
zig build run
# → mynotes listening on :8080
```

`--target=both` generates files for **both** native (`src/main.zig`) and Workers (`src/worker.zig`) entry points. The generated `main.zig` is a single-file demo of the modern model-based workflow — including a `Note` model, validators, schema migration, and CRUD handlers.

Test it:

```
curl -sS -X POST -H 'content-type: application/json' \
  -d '{"title":"hi","body":"first note"}' \
  http://127.0.0.1:8080/notes
curl -sS http://127.0.0.1:8080/notes
```

---

2. Models (3 minutes)

Models are plain Zig structs with a `pub const __schema` block. Akamata introspects this at comptime to build CREATE TABLE SQL, validators, queries, and migrations.

```
pub const User = struct {
    id: ?i64 = null,                    // ?i64 = null → INTEGER PRIMARY KEY AUTOINCREMENT
    email: []const u8,                  // TEXT NOT NULL
    name: []const u8,
    age: ?i32 = null,                   // INTEGER (nullable)
    created_at: ?i64 = null,            // INTEGER DEFAULT (unixepoch())

    pub const __schema = .{
        .table = "users",               // optional, default = lowercased struct name + "s"
        .primary_key = "id",            // optional, default = "id"
        .indexes = .{
            .{ "email", .unique },      // single-column unique index → email_unq
            .{ "name",  .index },       // non-unique
        },
        .defaults = .{
            .created_at = "unixepoch()",
        },
        .validates = .{
            .email = .{ am.model.rule.required, am.model.rule.max_len(255), am.model.rule.fo
            .name  = .{ am.model.rule.required, am.model.rule.max_len(80) },
            .age   = .{ am.model.rule.range(0, 150) },
        },
        // Optional: rename Zig fields → SQL columns
        // .columns = .{ .userId = "user_id" },
        // Optional: relations
        // .relations = .{
        //     .posts = .{ .has_many = .{ .model = Post, .fk = "user_id" } },
        // },
    };
};
```

Custom validators

```
fn requireAcmeDomain(value: []const u8, _: std.mem.Allocator) ?[]const u8 {
    return if (std.mem.endsWith(u8, value, "@acme.co")) null else "must be acme.co email";
}

pub const __schema = .{
    .validates = .{
        .email = .{ am.model.rule.required, am.model.rule.custom(requireAcmeDomain) },
    },
};
```

3. Repo: typed CRUD (2 minutes)

```
const Users = am.model.repo(User);

// Read
const u = try Users.find(c.db(), c.arena, 42);          // ?User
const all = try Users.all(c.db(), c.arena);             // []User (newest first)
const adults = try Users.where(c.db(), c.arena, .{ .age = 30 });

// Write
const created = try Users.create(c.db(), c.arena, .{ .email = "x@y", .name = "x" });
var u2 = created;
u2.name = "renamed";
try Users.save(c.db(), c.arena, &u2);
try Users.delete(c.db(), u2.id.?);

// Escape hatch — arbitrary SQL still mapped to User
const old = try Users.queryRaw(c.db(), c.arena,
    "SELECT id, email, name, age, created_at FROM users WHERE age > ? ORDER BY age DESC LIMI
    .{30},
);

// Eager loading (N+1 killer)
const owners = try Users.all(c.db(), c.arena);
const loaded = try am.model.preload.hasMany(User, "posts", owners, c.db(), c.arena);
for (loaded) |row| {
    // row.parent: User; row.related: []Post (one batched IN query)
}
```

4. Handlers: the short version (2 minutes)

```
const Ctx = am.Context(State);

fn createNote(c: *Ctx) !void {
    // Parse JSON + run __schema.validates. On bad JSON: writes 400 + returns null.
    // On validation failure: writes 422 + returns null. Either way, just early-return.
    const note = (try c.input(Note)) orelse return;
    const created = try Notes.create(c.db(), c.arena, note);
    try c.json(created, 201);
}

fn showNote(c: *Ctx) !void {
    const id = c.req.paramAs(i64, "id") catch return c.badRequest("invalid id");
    const n = (try Notes.find(c.db(), c.arena, id)) orelse return c.notFound();
    try c.json(n, 200);
}
```

Response shortcuts (all emit a consistent `{ error_kind, message? }`)

Method	Status	Use
`c.badRequest(msg)`	400	Malformed input
`c.unauthorized(msg)`	401	Missing/invalid token
`c.forbidden(msg)`	403	Auth'd but not allowed
`c.notFound()`	404	Resource gone
`c.conflict(msg)`	409	Duplicate / state mismatch
`c.unprocessable(errs)`	422	Validation failures
`c.serverError(msg)`	500	Generic server error
`c.json(value, code)`	any	Custom code

State / config shortcuts

```
pub const State = struct {
    db: am.db.Db,
    cfg: MyConfig,
};

fn handler(c: *Ctx) !void {
    _ = c.db();   // == c.state().db
    _ = c.cfg();  // == c.state().cfg
}
```

---

5. Routing + middleware (1 minute)

```
pub fn registerRoutes(app: *am.App(State)) !void {
    _ = try app.useAll(am.mw.recover(State));
    _ = try app.useAll(am.mw.logger(State));
    _ = try app.useAll(am.mw.requestId(State));         // adds X-Request-ID
    _ = try app.useAll(am.mw.accessLog(State, .json));  // structured access log
    _ = try app.useAll(am.mw.cors(State, .{}));         // CORS preflight
    _ = try app.useAll(am.mw.rateLimit(State, .{ .max = 100, .per_secs = 60 }));

    _ = try app.get("/notes", listNotes);
    _ = try app.post("/notes", createNote);
    _ = try app.get("/notes/:id", showNote);
    _ = try app.delete("/notes/:id", deleteNote);

    _ = try app.ws("/live", liveHandler);
}
```

---

6. Picking a backend with `DATABASE_URL` (1 minute)

Scheme	Backend	Where
`file:./mynotes.db`	SQLite	native only
`libsql://<db>.turso.io?authToken=…`	Turso (libsql HTTP)	native + Workers
`https://<db>.turso.io?authToken=…`	Turso (HTTPS alias)	native + Workers
`d1:DB`	Cloudflare D1 binding	Workers only

```
# Local SQLite (default)
zig build run

# Turso
DATABASE_URL='libsql://my.turso.io?authToken=eyJ…' zig build run
```

For Workers, set `DATABASE_URL` in `deploy/wrangler.toml`'s `[vars]` block.

---

7. Migrations (2 minutes)

There are two flavours, pick what fits:

A. Auto-diff (good for early dev)

The scaffold runs this on every native startup:

```
const plan = try am.model.migrate.diff(arena, db, &all_models);
try am.model.migrate.apply(arena, db, plan);
```

For Workers, use the `migrate_once.run` middleware (the scaffold wires it on first request — JSPI isn't available during `akamata_init`).

B. Versioned files (production-friendly)

```
akamata migrate generate add_users        # creates migrations/<ts>_add_users.sql
# ...edit the file with your SQL...
./zig-out/bin/myapp migrate-up            # applies pending, records in schema_migrations
```

`migrate-up` is a subcommand on your generated `main.zig` that wraps `am.model.migrate.Migrator`. Edit it to suit (custom locking, dry-run, etc.).

---

8. Deploy (3 minutes)

VPS / Container

```
zig build -Doptimize=ReleaseFast -Dtarget=x86_64-linux-musl
# zig-out/bin/myapp is now a static linux binary
```

`deploy/Dockerfile` (generated by `akamata init`) is `FROM scratch + COPY`. `docker build -f deploy/Dockerfile -t myapp .` and you're done.

Cloudflare Workers + D1

```
akamata deploy --workers \
  --config=deploy/wrangler.toml \
  --migrate=<(./zig-out/bin/myapp --print-schema)
```

One command:

1. Reads `wrangler.toml`'s `[[d1_databases]]` block
2. If `database_id` is the `00000000-...` placeholder, runs `wrangler d1 create <name>` and writes the real UUID back. If the DB already exists in your account, adopts that UUID via `wrangler d1 list --json`.
3. Applies the schema you supplied via `--migrate` to the remote D1.
4. `zig build -Dbackend=workers -Doptimize=ReleaseSmall`
5. `wrangler deploy --config=...`

`<(...)` is process substitution — pipes the schema directly without a temp file. Or use a normal file: `--migrate=migrations/00_init.sql`.

Cloudflare Workers + Turso

Same as D1, but in `wrangler.toml`:

```
[vars]
DATABASE_URL = "libsql://<db>.turso.io?authToken=…"
# remove [[d1_databases]]
```

then `akamata deploy --workers --config=deploy/wrangler.toml`. The wasm calls libsql/Hrana over `fetch()` through JSPI — same handler code.

---

9. Cheat sheet

```
akamata init <name> [--target=native|workers|containers|both]
akamata build [--workers|--containers]
akamata dev
akamata deploy [--workers|--containers] [--config=PATH] [--migrate=SQL]
akamata db <sql-file> [--local|--remote] [--config=PATH]
akamata migrate generate <name> [--dir=migrations]
akamata migrate up [--dir=migrations]
akamata --version
akamata help

# When typoed:
$ akamata deplyo
akamata: unknown subcommand `deplyo`
Did you mean `akamata deploy`?
```

```
# In handlers:
c.db()                       == c.state().db
c.cfg()                      == c.state().cfg
c.input(T)                   parse + validate → ?T (writes 400/422 on fail)
c.json(value, code)
c.badRequest / c.notFound / c.unauthorized / c.forbidden / c.conflict / c.unprocessable / c.
c.req.json(T)                raw parse (no validation)
c.req.paramAs(i64, "id")
c.req.query("q")
c.req.header("x-foo")
c.req.cookie("session")
c.setCookie(name, value, .{ .secure = true })
```

```
# Models:
am.model.tableDef(T)         comptime TableDef
am.model.repo(T)             Repo with find/all/where/create/save/delete/queryRaw
am.model.validate(T, v, arena)
am.model.migrate.diff/apply
am.model.migrate.Once        deferred guard for Workers
am.model.migrate.Migrator    versioned file runner
am.model.preload.hasMany(Owner, "rel", parents, db, arena)
am.model.relations.hasMany / belongsTo  (lazy)
```

Where to go next

- `examples/guestbook/` — minimum complete app with HTML UI, validation, all three DB backends
- `examples/mobus/` — bigger realistic app (auth, friends, messages, WS hub, FCM push)
- `examples/chat/` — Durable Object SQLite + WebSocket
- `docs/db-backends.md` — Backend implementation notes (JSPI, Hrana protocol)
- `docs/benchmarks.md` — Performance numbers (167k req/s on hot paths)
- `docs/architecture.md` — Framework internals + production hardening notes